Security Review Report

NM-0411-0817 - World
Nullifier Contracts

February 18, 2026

NETHERMIND
SECURITY

Contents

1 Executive Summary

This document presents the results of the security review conducted by Nethermind Security for TACEO's OPRF contracts. The OPRF Key Registry contract implements the three-round key-generation and resharing protocol described in A Nullifier Protocol based on a Verifiable, Threshold OPRF. It can run multiple instances of this protocol, tracking shares and commitments across rounds and ultimately deriving the corresponding public key once the protocol completes. **The audit comprises 1122** lines of the Solidity code. **The audit was performed using** (a) manual analysis of the codebase, and (b) automated analysis tools.

Along this document, we report 3 points of attention, where one is classified as `Medium`, and two are classified as `Low`. The issues are summarized in Fig. 1.

This document is organized as follows. Section 2 presents the files in the scope. Section 3 presents the summary of issues. Section 5 discusses the risk rating methodology. Section 6 details the issues. Section 7 discusses the documentation provided by the client for this audit. Section 8 presents the test suite evaluation and automated tools used. Section 9 concludes the document.

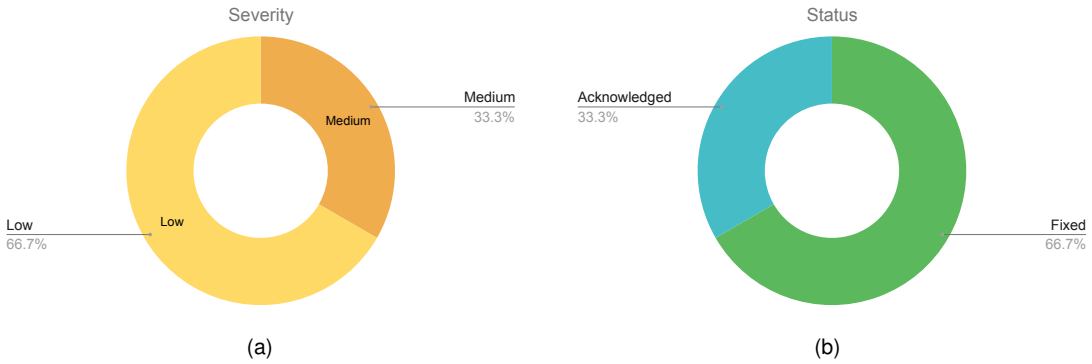

Fig. 1: Distribution of issues: Critical (0), **High** (0), **Medium** (1), **Low** (2), **Undetermined** (0), **Informational** (0), **Best Practices** (0).
Distribution of status: Fixed (2), **Acknowledged** (1), **Mitigated** (0), **Unresolved** (0)

Summary of the Audit

Audit Type	Security Review
Final Report	February 18, 2026
Initial Commit	`30eff1a`
Final Commit	`824e6bb`
Documentation Assessment	High
Test Suite Assessment	High

2 Audited Files

	Contract	LoC	Comments	Ratio	Blank	Total
1	src/OprfKeyGen.sol	114	53	46.5%	16	183
2	src/BabyJubJub.sol	456	145	31.8%	32	633
3	src/OprfKeyRegistry.sol	552	257	46.6%	81	890
	Total	**1122**	**455**	**40.6%**	**129**	**1706**

3 Summary of Issues

	Finding	Severity	Update
1	participants can reuse past rounds contributions from other parties	Medium	Fixed
2	Deleted `oprfKeyId` can be initiated again	Low	Fixed
3	`numPeers` should be exact	Low	Acknowledged

4 System Overview

The **OprfKeyRegistry** contract coordinates a multi-round MPC protocol among a fixed set of registered OPRF peers to generate, reshare, and register an OPRF public key (a BabyJubJub point) under a given OPRF key identifier (`oprfKeyId`). The contract is deployed behind an upgradeable proxy and initialized by the owner, who sets the Groth16 verifier address, the `(threshold, numPeers)` parameters, and an initial keygen admin. After the owner registers exactly `numPeers` distinct peer addresses (each assigned a stable `partyId`), the system is marked ready and key-generation sessions may begin.

Protocol state for each `oprfKeyId` is stored in `runningKeyGens[oprfKeyId]` (an `OprfKeyGen.OprfKeyGenState`), while finalized keys are stored in `oprfKeyRegistry[oprfKeyId]` as `RegisteredOprfPublicKey {key,epoch}`. A session can be started either as a fresh key generation (`initKeyGen`) or as a reshare (`initReshare`) for an existing key, which increments the epoch and re-randomizes / redistributes shares while preserving the registered public key identity.

Following the three-round structure used by the protocol, round progression is driven by peer submissions and contract-emitted events:

a. Round 1: Each registered peer submits a `Round1Contribution` containing a commitment share (`commShare`), a commitment to polynomial coefficients (`commCoeffs`), and an ephemeral BabyJubJub public key (`ephPubKey`).
   - In key-gen all peers are treated as PRODUCERS and must provide non-empty commitments; the contract aggregates `commShare` values into `keyAggregate`.
   - In reshare, peers may act as PRODUCER or CONSUMER. Producers must match their previous share commitment (prevShareCommitments) to prove consistent input, and once threshold producers are identified the contract computes and stores Lagrange coefficients for round 2.
   - When all peers have submitted round 1, the contract transitions to Round 2 (or marks the session STUCK if too few producers volunteered in a reshare) and emits SecretGenRound2 (key-gen) or the corresponding reshare round event.

b. Round 2: Only PRODUCERS submit a `Round2Contribution`, which contains encrypted share material for recipients (`SecretGenCiphertext[] ciphers`) and a compressed Groth16 proof (`compressedProof`). The contract:
   (a) enforces correct round and one-submission-per-party
   (b) validates BabyJubJub points in ciphertext commitments
   (c) accumulates shareCommitments (plain sum for key-gen; Lagrange-weighted accumulation for reshare)
   (d) verifies the Groth16 proof via a configured verifier (currently hard-coded public input layouts for (numPeers,threshold) = $(3,2)$ and $(5,3)$).

   Once all required producers have submitted (all peers for key-gen; `threshold` for reshare), the contract enters Round 3 and emits `SecretGenRound3` (key-gen) or `ReshareRound3` (reshare, also publishing Lagrange coefficients and epoch).

c. Round 3: All peers submit a final "done" acknowledgment (addRound3Contribution). When all parties have acknowledged, the contract finalizes:
   - For key-gen (epoch $== 0$), it registers `keyAggregate` as the OPRF public key for `oprfKeyId`.
   - For reshare, it updates only the stored epoch. In both cases it persists prevShareCommitments = shareCommitments for future reshares, emits `SecretGenFinalize`, and resets transient session state while keeping the key reusable.

The accompanying **OprfKeyGen.sol** file is a library that defines the protocol's shared types (round enums, role enums, contribution structs, ciphertext format, and the per-key `OprfKeyGenState`) plus helper routines to initialize, reset, and delete protocol state safely across key-gen and reshare sessions.

The **BabyJubJub contract** supplies the elliptic-curve arithmetic and validation used throughout (point addition, scalar multiplication, on-curve / subgroup checks, and Lagrange coefficient computation), ensuring deterministic, field-safe computations for all curve-based commitments and aggregates.

4.1 Access Control

Separates governance from participation. The owner registers the peer set and is the only party allowed to authorize upgrades (UUPS). Keygen admins (managed by `addKeyGenAdmin` and `revokeKeyGenAdmin`) are the only entities allowed to start, abort, delete keys, or trigger (re)initialization flows. Only registered peers may submit round contributions; submissions are enforced via strict `msg.sender → partyId` checks, round guards (`WrongRound`), and replay protection (`AlreadySubmitted`). Readiness is enforced by `isReady`, and proxy/initializer safety is enforced via `onlyProxy` and `onlyInitialized`.

5 Risk Rating Methodology

The risk rating methodology used by Nethermind Security follows the principles established by the OWASP Foundation. The severity of each finding is determined by two factors: **Likelihood** and **Impact**.

Likelihood measures how likely the finding is to be uncovered and exploited by an attacker. This factor will be one of the following values:

a) **High**: The issue is trivial to exploit and has no specific conditions that need to be met;

b) **Medium**: The issue is moderately complex and may have some conditions that need to be met;

c) **Low**: The issue is very complex and requires very specific conditions to be met.

When defining the likelihood of a finding, other factors are also considered. These can include but are not limited to motive, opportunity, exploit accessibility, ease of discovery, and ease of exploit.

Impact is a measure of the damage that may be caused if an attacker exploits the finding. This factor will be one of the following values:

a) **High**: The issue can cause significant damage, such as loss of funds or the protocol entering an unrecoverable state;

b) **Medium**: The issue can cause moderate damage, such as impacts that only affect a small group of users or only a particular part of the protocol;

c) **Low**: The issue can cause little to no damage, such as bugs that are easily recoverable or cause unexpected interactions that cause minor inconveniences.

When defining the impact of a finding, other factors are also considered. These can include but are not limited to Data/state integrity, loss of availability, financial loss, and reputation damage. After defining the likelihood and impact of an issue, the severity can be determined according to the table below.

		Severity Risk		
Impact	**High**	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Info/Best Practices	Low	Medium
	Undetermined	Undetermined	Undetermined	Undetermined
		Low	**Medium**	**High**
		Likelihood		

To address issues that do not fit a High/Medium/Low severity, Nethermind Security also uses three more finding severities: **Informational**, **Best Practices**, and **Undetermined**.

a) **Informational** findings do not pose any risk to the application, but they carry some information that the audit team intends to pass to the client formally;

b) **Best Practice** findings are used when some piece of code does not conform with smart contract development best practices;

c) **Undetermined** findings are used when we cannot predict the impact or likelihood of the issue.

6 Issues

6.1 [Medium] participants can reuse past rounds contributions from other parties

File(s): src/OprfKeyRegistry.sol

Description: The key generation process consists of three rounds in which all registered peers must participate. Round 1 requires submitting secret commitments and aggregating the key, Round 2 involves submitting zero-knowledge proofs of correct secret calculation, and Round 3 requires acknowledging the reception of ciphertexts.

The issue arises from the fact that nothing prevents a participant from submitting another party's round contribution as their own. Round 1 contributions only verify the validity of the commitment and ensure that the participant has not already contributed:

```
function addRound1KeyGenContribution(uint160 oprfKeyId, OprfKeyGen.Round1Contribution calldata data)
    public virtual onlyProxy isReady
{
    // return the partyId if sender is really a participant
    uint16 partyId = _internParticipantCheck();
    _curveChecks(data.commShare);
    if (data.commCoeffs == 0) revert BadContribution();
    OprfKeyGen.OprfKeyGenState storage st = _addRound1Contribution(oprfKeyId, partyId, data);
    // check that this is a key-gen
    if (st.generatedEpoch != 0) {
        revert BadContribution();
    }
    st.nodeRoles[msg.sender] = OprfKeyGen.KeyGenRole.PRODUCER;
    st.numProducers += 1;
    _addToAggregate(st.keyAggregate, data.commShare);
    _tryEmitRound2Event(oprfKeyId, numPeers, st);
        emit OprfKeyGen.KeyGenConfirmation(oprfKeyId, partyId, 1, st.generatedEpoch);
}
```

Round 2 contributions have similar checks, with the addition of zero-knowledge proof verification that embeds public data observable by other participants:

```
function addRound2Contribution(uint160 oprfKeyId, OprfKeyGen.Round2Contribution calldata data)
    public virtual onlyProxy isReady
{
    if (data.ciphers.length != numPeers) revert BadContribution();
        OprfKeyGen.OprfKeyGenState storage st = runningKeyGens[oprfKeyId];
    if (st.currentRound != OprfKeyGen.Round.TWO) revert WrongRound(st.currentRound);

    uint16 partyId = _internParticipantCheck();
    if (st.round2Done[partyId]) revert AlreadySubmitted();
    if (OprfKeyGen.KeyGenRole.PRODUCER != st.nodeRoles[msg.sender]) revert BadContribution();
    // --SNIP
    if (numPeers == 3 && threshold == 2) {
        IVerifierKeyGen13 keyGenVerifier13 = IVerifierKeyGen13(keyGenVerifier);

        uint256[PUBLIC_INPUT_LENGTH_KEYGEN_13] memory publicInputs;

        BabyJubJub.Affine[] memory pubKeyList = _loadPeerPublicKeys(st);
        publicInputs[0] = pubKeyList[partyId].x;
        publicInputs[1] = pubKeyList[partyId].y;
        publicInputs[2] = st.round1[partyId].commShare.x;
        publicInputs[3] = st.round1[partyId].commShare.y;
        publicInputs[4] = st.round1[partyId].commCoeffs;
        publicInputs[5 + (numPeers * 3)] = threshold - 1;
        for (uint256 i = 0; i < numPeers; ++i) {
            publicInputs[5 + i] = data.ciphers[i].cipher;
                publicInputs[5 + numPeers + (i * 2) + 0] = data.ciphers[i]commitment.x;
            publicInputs[5 + numPeers + (i * 2) + 1] = data.ciphers[i].commitment.y;
            publicInputs[5 + (numPeers * 3) + 1 + (i * 2) + 0] = pubKeyList[i].x;
            publicInputs[5 + (numPeers * 3) + 1 + (i * 2) + 1] = pubKeyList[i].y;
            publicInputs[5 + (numPeers * 5) + 1 + i] = data.ciphers[i].nonce;
        }
        keyGenVerifier13.verifyCompressedProof(data.compressedProof, publicInputs);
        }
    // --SNIP
```

```
}
```

As demonstrated above, neither function checks for duplicate contributions across different participants. This allows participants to replicate contributions from others, resulting in the final key being generated with fewer unique contributions than required. This vulnerability directly contradicts the specification outlined in the whitepaper this smart contract implements, specifically in *Figure 7: Proposal 2 for key generation using ZK proofs* says:

> The smart contract checks if $X_i = X_j$ for any $i \neq j$ in which case it aborts. Otherwise, it computes the public key from the commitments $pk = \prod_{i=1}^{t} X_i$.

Recommendation(s): Multiple approaches can address this issue. One solution, as suggested in the whitepaper, is to implement duplicate contribution detection. Alternatively, the `partyId` could be included as a public input in the zero-knowledge proofs to cryptographically bind each contribution to its participant.

Status: Fixed

Update from the client: Fixed in commit 824e6bb

6.2 [Low] Deleted `oprfKeyId` can be initiated again

File(s): src/OprfKeyRegistry.sol

Description: An admin can delete an existing `oprfKeyId` by calling `deleteOprfPublicKey`. Deleted OPRF keys are expected to remain permanently deleted, as both the `initKeyGen` and `initReshare` functions explicitly check against the `DELETED` state:

```
function initKeyGen(uint160 oprfKeyId) public virtual onlyProxy isReady onlyAdmin {
    if (oprfKeyId == 0) revert BadContribution();
    // Check that this oprfKeyId was not used already
    BabyJubJub.Affine storage publicKey = oprfKeyRegistry[oprfKeyId].key;
    OprfKeyGen.OprfKeyGenState storage st = runningKeyGens[oprfKeyId];

    // check if deleted
@==>    if (st.currentRound == OprfKeyGen.Round.DELETED) revert DeletedId(oprfKeyId);
    // -- SNIP
```

However, an admin can intentionally or unintentionally bypass this protection by calling `abortKeyGen` on a deleted `oprfKeyId`, which resets its state to `NOT_STARTED`, thereby allowing the key generation process to be reinitiated:

```
function abortKeyGen(uint160 oprfKeyId) public virtual onlyProxy isReady onlyAdmin {
    OprfKeyGen.OprfKeyGenState storage st = runningKeyGens[oprfKeyId];
    if (st.currentRound == OprfKeyGen.Round.NOT_STARTED) {
        revert UnknownId(oprfKeyId);
    }
    st.reset(numPeers, peerAddresses);
    emit OprfKeyGen.KeyGenAbort(oprfKeyId);
}

function reset(OprfKeyGenState storage st, uint256 numPeers, address[] memory peerAddresses) internal {
    _reset(st, numPeers, peerAddresses);
    st.currentRound = Round.NOT_STARTED;
}
```

Recommendation(s): Consider adding a validation check in the `abortKeyGen` function to prevent aborting operations on `oprfKeyId` entries that are in the `DELETED` state.

Status: Fixed

Update from the client: Fixed in commit 61f492c

6.3 [Low] `numPeers` should be exact

File(s): src/OprfKeyRegistry.sol

Description: The circuit using the number of peers requires it to be a fixed number but the initialiser allows for any value for the number of peers that is greater than 2.

The initialiser allows the passing of any `uint16` value as `threshold` or `numPeers` seen below:

```
/// @param _threshold The threshold number of peers required for key generation.
/// @param _numPeers The number of peers participating in the key generation.
function initialize(
    address _owner,
    address _keygenAdmin,
    address _keyGenVerifierAddress,
    uint16 _threshold,
    uint16 _numPeers
) public virtual initializer {
    // -- SNIP
    // @audit this should have a fixed number of 2 or 3 so it does not revert in round 2 contribution
    threshold = _threshold;
    // @audit this should have a fixed number of 3 or 5
    numPeers = _numPeers;
    isContractReady = false;
}
```

When performing the ZK proof verification and building the public inputs, there is a check that `numPeers` and `threshold` are either `(3,2)` or `(5,3)`; otherwise, the contract reverts:

```
if (numPeers == 3 && threshold == 2) {
    // -- SNIP
    keyGenVerifier13.verifyCompressedProof(data.compressedProof, publicInputs);
} else if (numPeers == 5 && threshold == 3) {
    // -- SNIP
    keyGenVerifier25.verifyCompressedProof(data.compressedProof, publicInputs);
} else {
    revert UnsupportedNumPeersThreshold();
}
```

This lack of a check allows the owner to incorrectly set either or both values, causing a Dos on all key generation.

Recommendation(s): Restrict peers and threshold to be the corresponding values required by the circuit.

Status: Acknowledged.

Update from the client: Acknowledged.

7 Documentation Evaluation

Software documentation refers to the written or visual information that describes the functionality, architecture, design, and implementation of software. It provides a comprehensive overview of the software system and helps users, developers, and stakeholders understand how the software works, how to use it, and how to maintain it. Software documentation can take different forms, such as user manuals, system manuals, technical specifications, requirements documents, design documents, and code comments. Software documentation is critical in software development, enabling effective communication between developers, testers, users, and other stakeholders. It helps to ensure that everyone involved in the development process has a shared understanding of the software system and its functionality. Moreover, software documentation can improve software maintenance by providing a clear and complete understanding of the software system, making it easier for developers to maintain, modify, and update the software over time. Smart contracts can use various types of software documentation. Some of the most common types include:

- Technical whitepaper: A technical whitepaper is a comprehensive document describing the smart contract's design and technical details. It includes information about the purpose of the contract, its architecture, its components, and how they interact with each other;
- User manual: A user manual is a document that provides information about how to use the smart contract. It includes step-by-step instructions on how to perform various tasks and explains the different features and functionalities of the contract;
- Code documentation: Code documentation is a document that provides details about the code of the smart contract. It includes information about the functions, variables, and classes used in the code, as well as explanations of how they work;
- API documentation: API documentation is a document that provides information about the API (Application Programming Interface) of the smart contract. It includes details about the methods, parameters, and responses that can be used to interact with the contract;
- Testing documentation: Testing documentation is a document that provides information about how the smart contract was tested. It includes details about the test cases that were used, the results of the tests, and any issues that were identified during testing;
- Audit documentation: Audit documentation includes reports, notes, and other materials related to the security audit of the smart contract. This type of documentation is critical in ensuring that the smart contract is secure and free from vulnerabilities.

These types of documentation are essential for smart contract development and maintenance. They help ensure that the contract is properly designed, implemented, and tested, and they provide a reference for developers who need to modify or maintain the contract in the future.

Remarks about World's documentation

`World's` team provided an overview of the main system components during the kick-off call with a detailed explanation of the intended functionalities and use cases of the system. Moreover, the team addressed all questions and concerns raised by the Nethermind Security team, providing valuable insights and a comprehensive understanding of the project's technical aspects.

8 Test Suite Evaluation

8.1 Tests Output

```
> forge test

Ran 2 tests for test/Groth16Verifier.t.sol:KeyGen13Groth16Verifier
[PASS] testCompressedProof() (gas: 354600)
[PASS] testUncompressedProof() (gas: 345131)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 10.13ms (14.70ms CPU time)

Ran 12 tests for test/OprfKeyRegistry.keygen.t.sol:OprfKeyRegistryKeyGenTest
[PASS] testAbortBeforeRound2() (gas: 9951773)
[PASS] testAbortBeforeRound3() (gas: 14700890)
[PASS] testAbortDuringRound1() (gas: 9152137)
[PASS] testAbortDuringRound2() (gas: 11418263)
[PASS] testAbortDuringRound3() (gas: 14710451)
[PASS] testAbortKeyGenBeforeRound1() (gas: 7622333)
[PASS] testDeleteAfterInitKeyGenShouldFail() (gas: 7573560)
[PASS] testDeleteBeforeRound1() (gas: 7573252)
[PASS] testDeleteBeforeRound2() (gas: 7573428)
[PASS] testDeleteBeforeRound3() (gas: 7573307)
[PASS] testKeyGen() (gas: 7569438)
[PASS] testKeyGenThenDelete() (gas: 7431631)
Suite result: ok. 12 passed; 0 failed; 0 skipped; finished in 318.16ms (996.41ms CPU time)

Ran 11 tests for test/BabyJubjub.t.sol:BabyJubJubTest
[PASS] testAddGeneratorToItself() (gas: 62960)
[PASS] testAddIdentity() (gas: 6519)
[PASS] testCurveChecks() (gas: 1427037)
[PASS] testGeneratorOnCurve() (gas: 362565)
[PASS] testIdentityPoint() (gas: 352725)
[PASS] testIsEmpty() (gas: 7700)
[PASS] testIsEqual() (gas: 10897)
[PASS] testLagrangeCoeffsDegree2() (gas: 406230)
[PASS] testLagrangeCoeffsDegree3() (gas: 406527)
[PASS] testScalarMul() (gas: 1732944)
[PASS] testThreeTimes() (gas: 834692)
Suite result: ok. 11 passed; 0 failed; 0 skipped; finished in 318.22ms (33.65ms CPU time)

Ran 14 tests for test/OprfKeyRegistry.admin.t.sol:OprfKeyRegistryTest
[PASS] testConstructedCorrectly() (gas: 4530609)
[PASS] testDeleteBeforeKeyGen() (gas: 31526)
[PASS] testInitKeyGenResubmit() (gas: 419786)
[PASS] testInitKeyGenRevokeRegisterAdmin() (gas: 463457)
[PASS] testInitKeyGenWithZero() (gas: 21754)
[PASS] testInitReshareBeforeKeyGen() (gas: 30767)
[PASS] testRegisterAdminTwice() (gas: 28241)
[PASS] testRegisterParticipants() (gas: 4730879)
[PASS] testRegisterParticipantsNotDistinct() (gas: 4533694)
[PASS] testRegisterParticipantsNotTACEO() (gas: 4534528)
[PASS] testRegisterParticipantsWrongNumberKeys() (gas: 4532058)
[PASS] testRevokeAdminThatIsNoAdmin() (gas: 30516)
[PASS] testRevokeLastAdmin() (gas: 32362)
[PASS] testUpdateParticipants() (gas: 112258)
Suite result: ok. 14 passed; 0 failed; 0 skipped; finished in 318.28ms (40.39ms CPU time)

Ran 2 tests for test/OprfKeyRegistryUpgrade.t.sol:OprfKeyRegistryUpgradeTest
[PASS] testOwnershipTransfer() (gas: 42680)
[PASS] testUpgrade() (gas: 19473925)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 388.28ms (284.15ms CPU time)

Ran 32 tests for test/OprfKeyRegistry.reshare.t.sol:OprfKeyRegistryReshareTest
[PASS] testAbortAfterKeyGenThenReshare() (gas: 14827211)
[PASS] testAbortAfterReshareThenReshare() (gas: 21641715)
[PASS] testAbortBeforeRound2() (gas: 9954940)
[PASS] testAbortBeforeRound3() (gas: 14704703)
[PASS] testAbortDuringKeyGenAndAddRound2Contribution() (gas: 28404036)
[PASS] testAbortDuringKeyGenAndAddRound3Contribution() (gas: 28399644)
[PASS] testAbortDuringKeyGenInitAgainAndAddRound2Contribution() (gas: 7590266)
[PASS] testAbortDuringKeyGenInitAgainAndAddRound3Contribution() (gas: 7586210)
```

```
[PASS] testAbortDuringKeyGenMultipleTimes() (gas: 7377779)
[PASS] testAbortDuringReshareMultipleTimes() (gas: 16044260)
[PASS] testAbortDuringRound1() (gas: 9155094)
[PASS] testAbortDuringRound2() (gas: 11421780)
[PASS] testAbortDuringRound3() (gas: 14714400)
[PASS] testAbortKeyGenBeforeRound1() (gas: 7624986)
[PASS] testAbortReshareBeforeRound1() (gas: 14498728)
[PASS] testAbortReshareBeforeRound2() (gas: 16421492)
[PASS] testAbortReshareBeforeRound3() (gas: 21193187)
[PASS] testAbortReshareDuringRound1() (gas: 16039044)
[PASS] testAbortReshareDuringRound2() (gas: 18196452)
[PASS] testAbortReshareDuringRound3() (gas: 21212935)
[PASS] testDeleteAfterInitKeyGenShouldFail() (gas: 7576177)
[PASS] testDeleteBeforeRound1() (gas: 7575693)
[PASS] testDeleteBeforeRound2() (gas: 7575913)
[PASS] testDeleteBeforeRound3() (gas: 7575770)
[PASS] testInitReshareResubmit() (gas: 7813021)
[PASS] testKeyGen() (gas: 7571699)
[PASS] testKeyGenThenDelete() (gas: 7434077)
[PASS] testReshare1() (gas: 14448144)
[PASS] testReshare1ThenDelete() (gas: 14237236)
[PASS] testReshare2() (gas: 21262829)
[PASS] testReshare2ThenDelete() (gas: 21051649)
[PASS] testReshareIsStuck() (gas: 22813306)
Suite result: ok. 32 passed; 0 failed; 0 skipped; finished in 388.48ms (4.23s CPU time)

Ran 6 test suites in 391.28ms (1.74s CPU time): 73 tests passed, 0 failed, 0 skipped (73 total tests)
```

8.2 Automated Tools

8.2.1 AuditAgent

The AuditAgent is an AI-powered smart contract auditing tool that analyses code, detects vulnerabilities, and provides actionable fixes. It accelerates the security analysis process, complementing human expertise with advanced AI models to deliver efficient and comprehensive smart contract audits. Available at https://app.auditagent.nethermind.io.

9 About Nethermind

Nethermind is a Blockchain Research and Software Engineering company. Our work touches every part of the web3 ecosystem - from layer 1 and layer 2 engineering, cryptography research, and security to application-layer protocol development. We offer strategic support to our institutional and enterprise partners across the blockchain, digital assets, and DeFi sectors, guiding them through all stages of the research and development process, from initial concepts to successful implementation.

We offer security audits of projects built on EVM-compatible chains and Starknet. We are active builders of the Starknet ecosystem, delivering a node implementation, a block explorer, a Solidity-to-Cairo transpiler, and formal verification tooling. Nethermind also provides strategic support to our institutional and enterprise partners in blockchain, digital assets, and decentralized finance (DeFi). In the next paragraphs, we introduce the company in more detail.

Blockchain Security: At Nethermind, we believe security is vital to the health and longevity of the entire Web3 ecosystem. We provide security services related to Smart Contract Audits, Formal Verification, and Real-Time Monitoring. Our Security Team comprises blockchain security experts in each field, often collaborating to produce comprehensive and robust security solutions. The team has a strong academic background, can apply state-of-the-art techniques, and is experienced in analyzing cutting-edge Solidity and Cairo smart contracts, such as ArgentX and StarkGate (the bridge connecting Ethereum and StarkNet). Most team members hold a Ph.D. degree and actively participate in the research community, accounting for 240+ articles published and 1,450+ citations in Google Scholar. The security team adopts customer-oriented and interactive processes where clients are involved in all stages of the work.

Blockchain Core Development: Our core engineering team, consisting of over 20 developers, maintains, improves, and upgrades our flagship product - the Nethermind Ethereum Execution Client. The client has been successfully operating for several years, supporting both the Ethereum Mainnet and its testnets, and now accounts for nearly a quarter of all synced Mainnet nodes. Our unwavering commitment to Ethereum's growth and stability extends to sidechains and layer 2 solutions. Notably, we were the sole execution layer client to facilitate Gnosis Chain's Merge, transitioning from Aura to Proof of Stake (PoS), and we are actively developing a full-node client to bolster Starknet's decentralization efforts. Our core team equips partners with tools for seamless node set-up, using generated docker-compose scripts tailored to their chosen execution client and preferred configurations for various network types.

DevOps and Infrastructure Management: Our infrastructure team ensures our partners' systems operate securely, reliably, and efficiently. We provide infrastructure design, deployment, monitoring, maintenance, and troubleshooting support, allowing you to focus on your core business operations. Boasting extensive expertise in Blockchain as a Service, private blockchain implementations, and node management, our infrastructure and DevOps engineers are proficient with major cloud solution providers and can host applications in-house or on clients' premises. Our global in-house SRE teams offer 24/7 monitoring and alerts for both infrastructure and application levels. We manage over 5,000 public and private validators and maintain nodes on major public blockchains such as Polygon, Gnosis, Solana, Cosmos, Near, Avalanche, Polkadot, Aptos, and StarkWare L2. Sedge is an open-source tool developed by our infrastructure experts, designed to simplify the complex process of setting up a proof-of-stake (PoS) network or chain validator. Sedge generates docker-compose scripts for the entire validator set-up based on the chosen client, making the process easier and quicker while following best practices to avoid downtime and being slashed.

Cryptography Research: At Nethermind, our cryptography Research team conducts cutting-edge internal research and collaborates closely with external partners on cryptographic protocols, consensus design, succinct arguments and folding schemes, elliptic curve-based STARK protocols, post-quantum security and zero-knowledge proofs (ZKPs). Our research has led to influential contributions, including Zinc (Crypto '25), Mova, FLI (Asiacrypt '24), and foundational results in Fiat-Shamir security and STARK proof batching. Complementing this theoretical work, our engineering expertise is demonstrated through implementations such as the Latticefold aggregation scheme, the Labrador proof system, zkvm-benchmarks, and Plonk Verifier in Cairo. This combined strength in theory and engineering enables us to deliver cutting-edge cryptographic solutions to partners and clients.

Smart Contract Development & DeFi Research: Our smart contract development and DeFi research team comprises 40+ world-class engineers who collaborate closely with partners to identify needs and work on value-adding projects. The team specializes in Solidity and Cairo development, architecture design, and DeFi solutions, including DEXs, AMMs, structured products, derivatives, and money market protocols, as well as ERC20, 721, and 1155 token design. Our research and data analytics focuses on three key areas: technical due diligence, market research, and DeFi research. Utilizing a data-driven approach, we offer in-depth insights and outlooks on various industry themes.

Our suite of L2 tooling: Warp is Starknet's approach to EVM compatibility. It allows developers to take their Solidity smart contracts and transpile them to Cairo, Starknet's smart contract language. In the short time since its inception, the project has accomplished many achievements, including successfully transpiling Uniswap v3 onto Starknet using Warp.

- **Voyager** is a user-friendly Starknet block explorer that offers comprehensive insights into the Starknet network. With its intuitive interface and powerful features, Voyager allows users to easily search for and examine transactions, addresses, and contract details. As an essential tool for navigating the Starknet ecosystem, Voyager is the go-to solution for users seeking in-depth information and analysis;
- **Horus** is an open-source formal verification tool for StarkNet smart contracts. It simplifies the process of formally verifying Starknet smart contracts, allowing developers to express various assertions about the behavior of their code using a simple assertion language;

- **Juno** is a full-node client implementation for Starknet, drawing on the expertise gained from developing the Nethermind Client. Written in Golang and open-sourced from the outset, Juno verifies the validity of the data received from Starknet by comparing it to proofs retrieved from Ethereum, thus maintaining the integrity and security of the entire ecosystem.

General Advisory to Clients

As auditors, we recommend that any changes or updates made to the audited codebase undergo a re-audit or security review to address potential vulnerabilities or risks introduced by the modifications. By conducting a re-audit or security review of the modified codebase, you can significantly enhance the overall security of your system and reduce the likelihood of exploitation. However, we do not possess the authority or right to impose obligations or restrictions on our clients regarding codebase updates, modifications, or subsequent audits. Accordingly, the decision to seek a re-audit or security review lies solely with you.

Disclaimer

This report is based on the scope of materials and documentation provided by you to Nethermind in order that Nethermind could conduct the security review outlined in **1. Executive Summary** and **2. Audited Files**. The results set out in this report may not be complete nor inclusive of all vulnerabilities. Nethermind has provided the review and this report on an as-is, where-is, and as-available basis. You agree that your access and/or use, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk. Blockchain technology remains under development and is subject to unknown risks and flaws. The review does not extend to the compiler layer, or any other areas beyond the programming language, or other programming aspects that could present security risks. This report does not indicate the endorsement of any particular project or team, nor guarantee its security. No third party should rely on this report in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset. To the fullest extent permitted by law, Nethermind disclaims any liability in connection with this report, its content, and any related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. Nethermind does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party through the product, any open source or third-party software, code, libraries, materials, or information linked to, called by, referenced by or accessible through the report, its content, and the related services and products, any hyperlinked websites, any websites or mobile applications appearing on any advertising, and Nethermind will not be a party to or in any way be responsible for monitoring any transaction between you and any third-party providers of products or services. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate. FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.